Harish B.K

Python – Intermediate Activity Handouts

Activity I1 – Problem Decomposition with Functions

Title: Breaking a Bigger Problem into Small Functions

Context
You now know variables, loops, if/else, and simple functions.
Next: learn to split a problem into clear sub-tasks.

Problem
Given a list of student marks, you must:

- Drop the lowest mark.
- Compute the new average.
- Decide pass/fail using a threshold.

Part A – Think in parts

1. Write the overall problem in one sentence:
   List of student marks.
2. Identify at least three smaller tasks:
   - Task 1: Drop lowest marks
   - Task 2: Calculate average.
   - Task 3: check pass / fail.
3. For each task, write a function name and its input and output:
   - Function 1 name: Drop lowest marks (60, 55, 70)
     - Input(s): list of marks (60, 55, 70)
     - Output: List after removing lowest marks = 60, 70
   - Function 2 name: Average
     - Input(s): list of marks 60, 55, 70
     - Output: average value is 61.66
   - Function 3 name: Pass
     - Input(s): average, threshold
     - Output: True / False

Part B – Implement functions

Fill and complete:

python

```
def drop_lowest(marks):
    # TODO: remove the smallest mark and return the new list
    return marks


def average(marks):
    total = 0
```

```
    for m in marks:
        total = total + m
    return total / len(marks)

def is_pass(avg, threshold):
    # return True or False
    if avg > Threshold
        return True
    else:
        return False

# Main flow
marks_list = [70, 85, 60, 90]
threshold = 50

new_marks = drop_lowest(marks_list)
avg = average(new_marks)
result = is_pass(avg, threshold)
print("Marks after dropping lowest:", new_marks)
print("Average:", avg)
print("Pass?", result)
```

Reflection

- How did dividing into functions help you think more clearly?

Dividing into functions makes the code easier to understand, fast & easy

Activity I2 – Working with Dictionaries

Title: Simple Student Record System

Problem
You need to manage student data: name, roll number, and marks.
You will use dictionaries for this.

Part A – Design

1. Decide a structure for one student record using a dict:

python

```
student = {
  "roll": 82,
  "name": "Harish",
  "marks": [70, 80, 90]
}
```

2. What keys will you use?
   - Key 1: roll
   - Key 2: name
   - Key 3: Marks.

Part B – Code

Complete:

python

```
students = []

# Add 3 students
for i in range(3):
  roll = int(input("Enter roll: "))
  name = input("Enter name: ")
  m1 = int(input("Mark 1: "))
  m2 = int(input("Mark 2: "))
  m3 = int(input("Mark 3: "))
  record = {
    "roll": roll,
    "name": name,
    "marks": [m1, m2, m3]
  }
  students.append(record)
```

```
# Print each student name and average mark
for s in students:
    marks = s["marks"]
    avg = sum(marks) / len(marks)
    print(s["roll"], s["name"], "Average:", avg)
```

Extension (optional)

- Add a decision: print “Pass” if average ≥ 50 else “Fail”.

```
if avg ≥ 50:
print ("pass")
else:
print ("fail")
```

Harish R/K

Activity I3 – Reading and Writing Files

Title: Process Data from a Text File

Problem

You are given a text file marks.txt with one mark per line.
You must read it and write a summary to summary.txt.

Part A – Understand

1. Inputs (file/content): marks.txt (Marks data)
2. Outputs (file/content): Summary.txt (Statistics summary)

Part B – Code Skeleton

python

```
# Step 1: Read marks from file
marks = []
with open("marks.txt", "r") as f:
    for line in f:
        line = line.strip()
        if line != "":
            marks.append(int(line))
# Step 2: Compute statistics
total = sum(marks)
count = len(marks)
average = total / count
max_mark = max(marks)
min_mark = min(marks)
# Step 3: Write summary to new file
with open("summary.txt", "w") as f:
    f.write("Number of students: " + str(count) + "\n")
    f.write("Average mark: " + str(average) + "\n")
    f.write("Max mark: " + str(max_mark) + "\n")
    f.write("Min mark: " + str(min_mark) + "\n")
```

count_60 =>
for mrk marks:
if m>=60:
count_60 += 1

Tasks

- Add one more line to summary: count of students with mark ≥ 60.
- Write that line in the file.

Reflection

- How is processing from file different from input()?

The input processes stored data, unlike input() which takes live user input

Activity I4 – Using Libraries (math / statistics)

Title: Use Standard Library for Cleaner Code

Problem

You need to analyze a list of numbers with mean, median, and standard deviation.

Part A – Manual vs Library

1. In simple words, what do these mean:
   - mean: Average of numbers
   - median: middle value when sorted
2. Write plain Python code (no extra library) to compute mean.

python

```
numbers = [10, 20, 30, 40, 50]

# Your mean logic here
```

mean = sum(~~num bers~~ numbers) / len(numbers)
print (mean)

Part B – Use statistics module

python

```
import statistics as stats


numbers = [10, 20, 30, 40, 50]

mean_value = stats.mean(numbers)

median_value = stats.median(numbers)

stdev_value = stats.pstdev(numbers)


print("Mean:", mean_value)

print("Median:", median_value)

print("Std Dev:", stdev_value)
```

Task

- Add code to ask the user for numbers (comma-separated) and then compute these three values using the library.

Activity I5 – Problem Template with Complexity Awareness

Title: Compare Two Approaches (Efficiency)

Problem

You must check if a number is present in a very large list (e.g., 1 million items).

Part A – Two Approaches

Approach 1: Simple loop search.

Approach 2: Use a set.

1. Describe Approach 1 in words:

loop through each element until the target is found

2. Describe Approach 2 in words:

Convert list to set & use fast membership checking

Part B – Implement Both

python

```
import time

# Create large list
nums = list(range(1000000))
target = 999999

# Approach 1: List search
start = time.time()
found = False
for x in nums:
    if x == target:
        found = True
        break
end = time.time()
print("List search found:", found, "Time:", end - start)

# Approach 2: Set search
num_set = set(nums)
start = time.time()
found2 = (target in num_set)
end = time.time()
print("Set search found:", found2, "Time:", end - start)
```

Reflection

- Which was faster and why (in simple words)?

Set search is faster because it uses hashing

Activity I6 – Input Validation and Error Handling

Title: Robust User Input

Problem

You want a safe function that always returns a valid integer entered by the user.

Part A – Requirements

1. What should happen if user types "abc"?

   Show error and ask again

2. What should happen if user leaves it blank?

   Show error & ask again

Part B – Code with try/except

python

```python
def get_int(prompt):
  while True:
    user_input = input(prompt)
    try:
      value = int(user_input)
      return value
    except ValueError:
      print("Invalid input. Please enter an integer.")


age = get_int("Enter your age: ")
print("Your age is:", age)
```

Tasks

- Modify get_int to also enforce a range (e.g., 0 to 120).
- If user enters out-of-range value, show a message and ask again.

```
def get_int(prompt):
while True:
Try:
  value = int(input (prompt))
    if 0<= value <= 120:
      return value
  else:
    print ("enter a value b/w 0 & 120")
    except value error:
    print( "invalid input, please enter an integer")
  age = get_int( "enter your age:")
  print ("your age is:", age)
```

Activity I7 – Mini Project: Text Analyzer

Title: Word Frequency Counter

Problem

Given a short paragraph, count how many times each word appears.

Part A – Plan

Use the problem template (short version):

1. Inputs: text Paragraph
2. Outputs: word frequency
3. Steps (bullet points):
   - Convert to lowercase
   - Split into words
   - count using dictionary.

Part B – Implement

python

```
text = input("Enter a sentence/paragraph: ")

# Convert to lower case
text = text.lower()

# Split into words
words = text.split()

# Count frequencies using dictionary
freq = {}
for w in words:
    if w in freq:
        freq[w] += 1
    else:
        freq[w] = 1

# Print results
for word, count in freq.items():
    print(word, ":", count)
```

Extensions (optional)

- Remove punctuation before counting.

Activity I8 – Applying the Full Problem-Solving Workflow

Title: Generic Approach Practice – Loan EMI Calculator

Problem

Compute the Equated Monthly Instalment (EMI) for a loan given:

- Principal (P)
- Annual interest rate (R, in %)
- Tenure in years (N)

Formula (you may give to students):

$$EMI = \frac{P \cdot r \cdot (1+r)^n}{(1+r)^n - 1}$$

where

- $r = R/(12 \times 100)$ (monthly rate)
- $n = N \times 12$ (months)

Use Full Template

1. Problem understanding
   - Inputs: Principle, Rate, years.
   - Outputs: EMI
   - Constraints: Positive value only
2. Analysis (steps)
   - Read P, R, N.
   - Convert Annual interest rate to monthly interest rate
   - Convert tenure from years to mathly
   - Apply EMI formula
   - Display the calculated EMI
3. Design
   - Functions: e.g., compute_emi(P, R, N)? Yes / No
   - Data types: float/int? P - float, R - float, N - int, EMI - float
4. Code

python

```
def compute_emi(P, R, N):
    r = R / (12*100)
    n = N*12
    emi = P*r* (1+r)**n / ((1+r)**n-1)
    return emi


P = float(input("Principal: "))
R = float(input("Annual interest rate (%): "))
N = int(input("Tenure (years): "))
```

```
emi_value = compute_emi(P, R, N)
print("Monthly EMI =", emi_value)
```

5. Test

- Test 1: P = 100000 R = 10, N = 1 → EMI = 8791
- Test 2: P = 500000 R = 8, N = 5 → EMI = 10138

Reflection

- Which step (understand/analyse/design/code/test) helped you most?

The analysis & design steps helped the most.